Alex Hendrix
Charlie Miner

Project 4: K-nearest Neighbors

Repository:
https://github.com/Churro1/CS5830_Project4_k-nearest-neighbors
Slides:
https://docs.google.com/presentation/d/1FoXgmdDHOkCtuQMFpxi-i9bNS7wSjAwiDl4-6PiPc5k/edit?usp=sharing

Part I

Introduction

The goal of this project was to use a K-nearest-neighbor model to predict whether a person has heart disease. We tested different model configurations, selected the one with the highest mean F1 score, and reported its results. The dataset contains information from heart-disease screenings conducted at Cleveland Clinic healthcare locations.

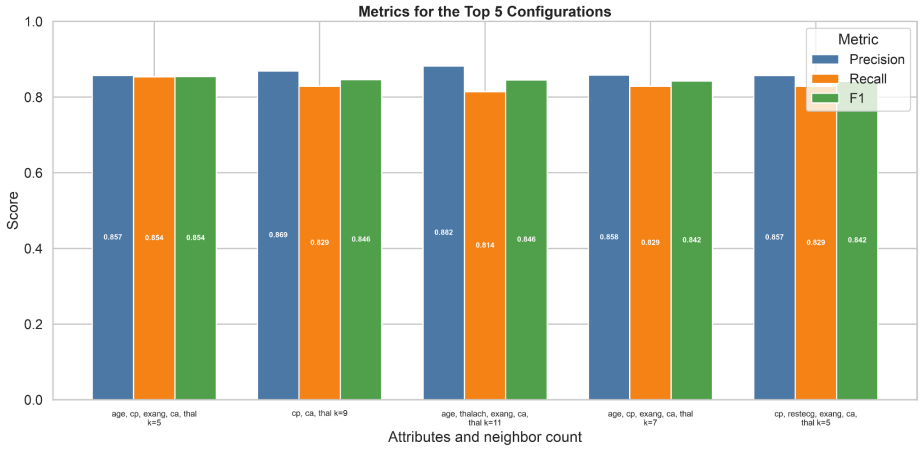
Methods

To determine the attributes and k value for our K-nearest-neighbor model, we tested six k values: 1, 3, 5, 7, 9, and 11. We generated every possible combination of 3, 4, or 5 predictor attributes, excluding the target columns. For each attribute combination and k value, we ran the model 10 times. Each run trained on 80% of the data and evaluated it on the remaining 20%. We then calculated the mean F1 score for each configuration. The configuration with the highest mean F1 score was selected as our best model.

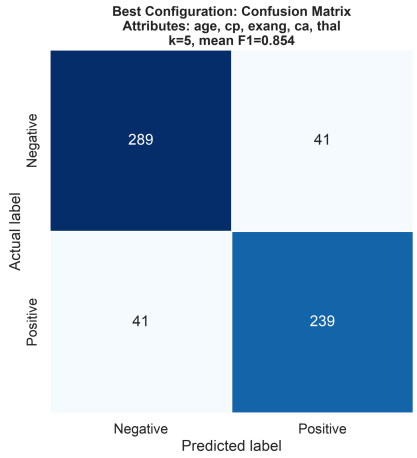
This search generated 2,288 feature subsets and 13,728 unique subset-and-k configurations. With 10 runs per configuration, we recorded 137,280 model evaluations. The feature subsets included 286 three-attribute subsets (17,160 model evaluations), 715 four-attribute subsets (42,900 model evaluations), and 1,287 five-attribute subsets (77,220 model evaluations).

Results

The top F1 score using the methods described above were the attributes: (age, cp, exang, ca, thal) with k=5.



Looking at the top 5 configurations, the F1, precision, and recall between different models are within a few percentage points. For the best configuration, we had a precision of 85.7%, recall of 85.4%, and a F1 score of 85.4%. Looking at the top 5 configurations, we can see that the attributes ca and thal are represented in every configuration, this suggests that these attributes could be important to predicting heart-disease.



Since we tested a large number of configurations with a moderate level of accuracy, we can be confident that we make reasonable diagnosis based on datasets similar to the one trained against.

Part II

Introduction

The goal of this project was to estimate the overall BoardGameGeek (BGG) rank of a board game based on its sub-category rankings (such as strategy, thematic, family, and party games). By treating this as a regression problem, we aimed to determine if a game's performance in specific niches is a reliable indicator of its overall popularity and critical reception among hobbyists.

Dataset

We used a dataset of board game rankings sourced directly from BoardGameGeek. To accomplish this, we found an existing database that has the overall game rank, and also sub-category rankings. We attempted to pull the full dataset (which includes much more information about each game) but it requires the devs to approve our api request and they did not do so in a timely manner. Our data set has 181,991 games.

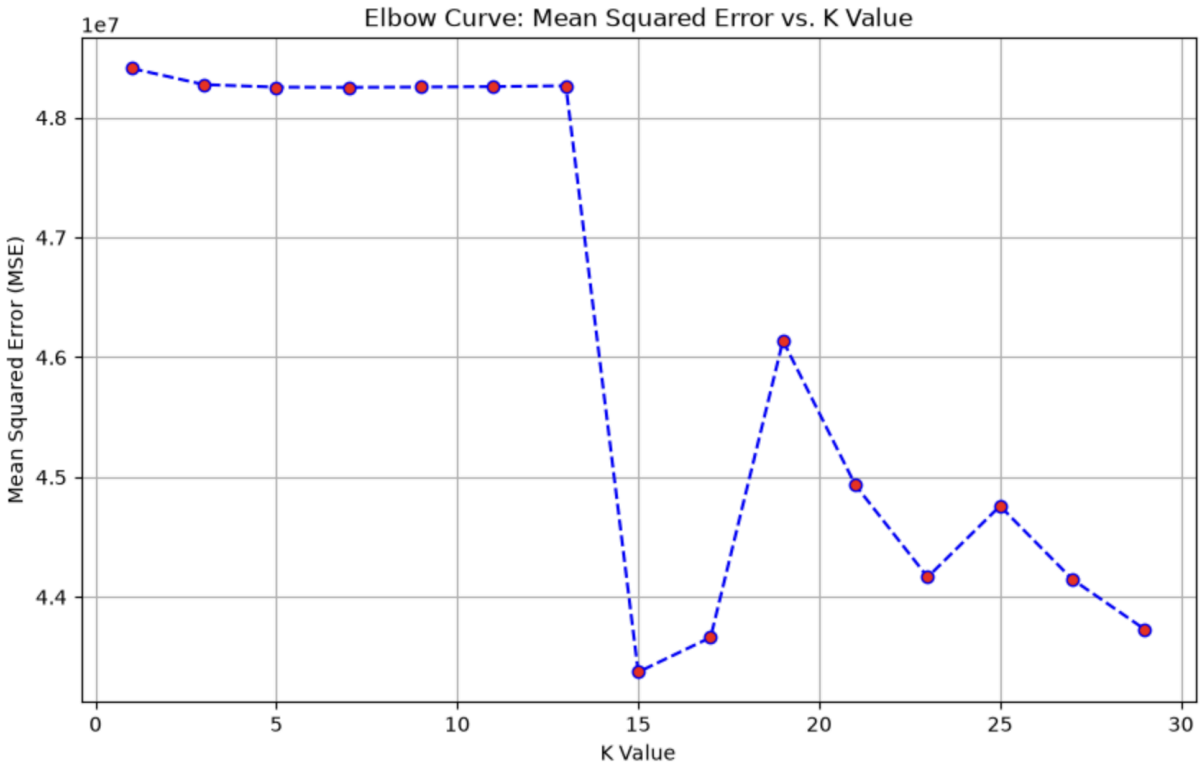
During the data cleaning process, we filtered the dataset to exclude expansions (is_expansion == 0), ensuring our model only evaluated standalone base games. For our features, we isolated the columns ending in _rank to capture the category-specific rankings. Because most games only belong to a few categories and are unranked in the rest, we had to handle significant missing data. We chose to impute these missing values with an arbitrarily large number (10,000,000). In the context of a distance-based algorithm, this effectively pushes unranked categories infinitely far away, preventing them from falsely implying similarity.

Methods

We implemented a k-nearest neighbors (KNN) regression model (KNeighborsRegressor via scikit-learn) utilizing a Euclidean distance metric. To evaluate performance and prevent overfitting, we performed an 80/20 train-test split.

To find the optimal number of neighbors (k), we tested a range of odd k values from 1 to 29. We calculated the Mean Squared Error (MSE) for each iteration and visualized the results using an elbow curve. Based on where the error rate stabilized on the curve, we

selected k=13 as our optimal hyperparameter for the final model.



Results

Using our optimized model (k=13), we achieved a final Mean Squared Error (MSE) of 48,265,572.32 on the test set. While this error metric is extremely large, this is expected given the vast range of board game ranks (spanning from 1 to over 100,000).

Despite the high MSE, the model proved surprisingly capable at estimating ranks for highly popular games. Testing the estimator on known data points yielded strong ballpark predictions:

- *Brass: Birmingham* (Actual: 1) was estimated at 12.
- *Agricola* (Actual: 66) was estimated at 83.
- *Cthulhu: Death May Die* (Actual: 55) was estimated at 83.
- *Wingspan* (Actual: 38) was estimated at 143.

	Game	Estimated Rank	Actual Rank	Off By
0	Brass: Birmingham	12	1	11
1	Wingspan	143	38	105
2	The Crew: Mission Deep Sea	143	45	98
3	Clank!: Catacombs	47	46	1
4	Harmonies	357	51	306
5	Cthulhu: Death May Die	83	55	28
6	Agricola	83	66	17
7	Obsession	61	67	6
8	Pandemic Legacy: Season 2	61	74	13
9	Pandemic	237	175	62
10	Flip 7	533	514	19

Overall, while predicting a game's exact numerical rank is difficult due to the high variance in the dataset, the KNN model successfully clusters games with similar sub-category prestige to provide a reasonable approximation of their overall standing.